Two-Factor Authentication (2FA) Working Model

Ekansh Sai Manikyala 2023002385
Mora Manaswik Reddy 2023003468

Table Of Content:

Abstract

Two-Factor Authentication (2FA) is a security mechanism that improves account protection by requiring two different forms of authentication before granting access. Traditional authentication systems rely only on passwords, which can easily be compromised through phishing attacks, brute force attacks, or password leaks.

This project demonstrates a working model of a Two-Factor Authentication system developed using HTML, CSS, and JavaScript. The model simulates a secure login system where a user must first enter a password and then verify a One-Time Password (OTP).

The OTP is randomly generated by the system and must be entered correctly to gain access. This model helps in understanding the concept of multi-layer security used in modern systems such as online banking, Gmail, and social media platforms. The project highlights how an additional authentication factor significantly improves cybersecurity.

Introduction

Cybersecurity has become one of the most important aspects of modern computing systems. As more services move online, protecting user data and preventing unauthorized access has become a major challenge.

Most traditional systems use password-based authentication. However, passwords alone are not secure because they can be guessed, stolen, or cracked using automated tools. Hackers often use techniques such as brute-force attacks, phishing, and credential stuffing to gain access to user accounts.

To overcome these security risks, many modern systems implement Two-Factor Authentication (2FA). In this method, users must provide two different authentication factors before accessing the system. This significantly reduces the chances of unauthorized access.

This project focuses on designing and implementing a simple working model that demonstrates how Two-Factor Authentication works using basic web technologies.

Objectives

- To understand the concept of authentication in cybersecurity.
- To study the working principle of Two-Factor Authentication.
- To develop a simple login system that uses OTP verification.
- To demonstrate how OTP improves the security of authentication systems.
- To create a beginner-friendly model for understanding multi-layer security.

Methodology

The development of the Two-Factor Authentication model was completed in several stages:

System Design:
A login interface was designed where the user must enter a password and then verify an OTP.

Technology Selection:
The project was implemented using web technologies including HTML, CSS, and JavaScript.

OTP Generation:
A random OTP is generated using JavaScript once the correct password is entered.

OTP Verification:
The user must enter the generated OTP to complete the login process.

User Interface Development:
A simple and visually appealing login interface was created with styled inputs and buttons.

System Architecture

The system architecture consists of three main components:

User Interface:
The login page where users enter credentials and OTP.

Authentication Module:
The JavaScript logic that verifies password and generates OTP.

Verification Module:
The system checks whether the entered OTP matches the generated OTP before granting access.

Architecture Flow:
User → Login Page → Password Verification → OTP Generation → OTP Verification → Access Granted

Flowchart of 2FA Process

1. Start
2. User enters username and password
3. System verifies password
4. If password incorrect → Show error
5. If password correct → Generate OTP
6. Display OTP input field
7. User enters OTP
8. System verifies OTP
9. If OTP correct → Login successful
10. End

Implementation

The working model simulates a login system with two levels of authentication.

Step 1: User enters username and password.
Step 2: System verifies password.
Step 3: If password is correct, a random OTP is generated.
Step 4: User enters OTP.
Step 5: System verifies OTP.
Step 6: If OTP matches, the user is successfully logged in.

JavaScript was used to generate OTP using the formula:
Math.floor(1000 + Math.random()*9000)

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<title>Two Factor Authentication Demo</title>

<link href="https://fonts.googleapis.com/css2?
family=Poppins:wght@300;400;600&display=swap" rel="stylesheet">

<style>

* {

margin:0;

padding:0;

box-sizing:border-box;

font-family:'Poppins',sans-serif;

}

body{

height:100vh;

display:flex;

justify-content:center;

align-items:center;

background:linear-gradient(135deg,#4facfe,#00f2fe); }

.container{

background:white;

padding:40px;

width:350px;

border-radius:12px;

box-shadow:0 10px 25px rgba(0,0,0,0.2);

text-align:center;

}

h2{

margin-bottom:20px;

color:#333;
```

```
}
input{
width:100%;
padding:10px;
margin:10px 0;
border:1px solid #ccc;
border-radius:6px;
font-size:14px;
}
button{
width:100%;
padding:10px;
background:#4facfe;
border:none;
color:white;
font-size:16px;
border-radius:6px;
cursor:pointer;
transition:0.3s;
}
button:hover{
background:#0077ff;
}
#otpSection{
display:none;
margin-top:10px;
}
#otpDisplay{
margin-top:10px;
font-weight:600;
color:#0077ff;
}
```

```
.success{
color:green;
font-weight:600;
margin-top:10px;
}
.error{
color:red;
margin-top:10px;
}
</style>
<script>
let generatedOTP = 0;
const correctPassword = “1234";
function login(){
let password = document.getElementById(“password").value.trim();
let message = document.getElementById(“message");
if(password === correctPassword){
generatedOTP = Math.floor(1000 + Math.random() * 9000);
document.getElementById("otpSection").style.display = “block";
document.getElementById("otpDisplay").innerText =
"Demo OTP: " + generatedOTP;
message.innerText = "Password Correct. Enter OTP.”;
message.className = “";
}else{
message.innerText = "Wrong Password!”;
message.className = “error";
document.getElementById("otpSection").style.display = “none";
}
}
function verifyOTP(){
let userOTP = document.getElementById("otp").value.trim();
```

```
let message = document.getElementById("message");
if(userOTP === generatedOTP.toString()){
message.innerText = "Login Successful ✔";
message.className = "success";
}else{
message.innerText = "Invalid OTP!";
message.className = "error";
}
}
</script>
</head>
<body>
<div class="container">
<h2>Two-Factor Authentication</h2>
<input type="text" placeholder="Username">
<input type="password" id="password" placeholder="Password">
<button type="button" onclick="login()">Login</button>
<div id="otpSection">
<p id="otpDisplay"></p>
<input type="text" id="otp" placeholder="Enter OTP">
<button type="button" onclick="verifyOTP()">Verify OTP</button>
</div>
<p id="message"></p>
</div>
</body>
</html>
```

Advantages of Two-Factor Authentication

- Provides stronger security compared to password-only systems.
- Reduces risk of unauthorized access.
- Protects sensitive user data.
- Widely used in banking and online services.
- Easy to implement with modern technologies.

Limitations

- Requires additional authentication step.
- OTP delivery may sometimes be delayed in real systems.
- Users must have access to their registered device.

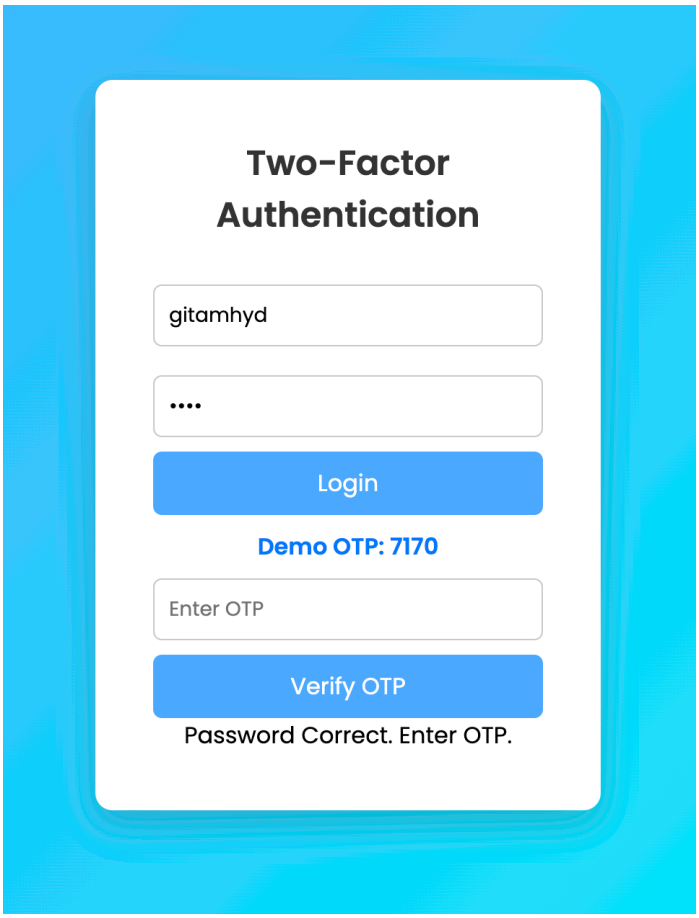
Applications

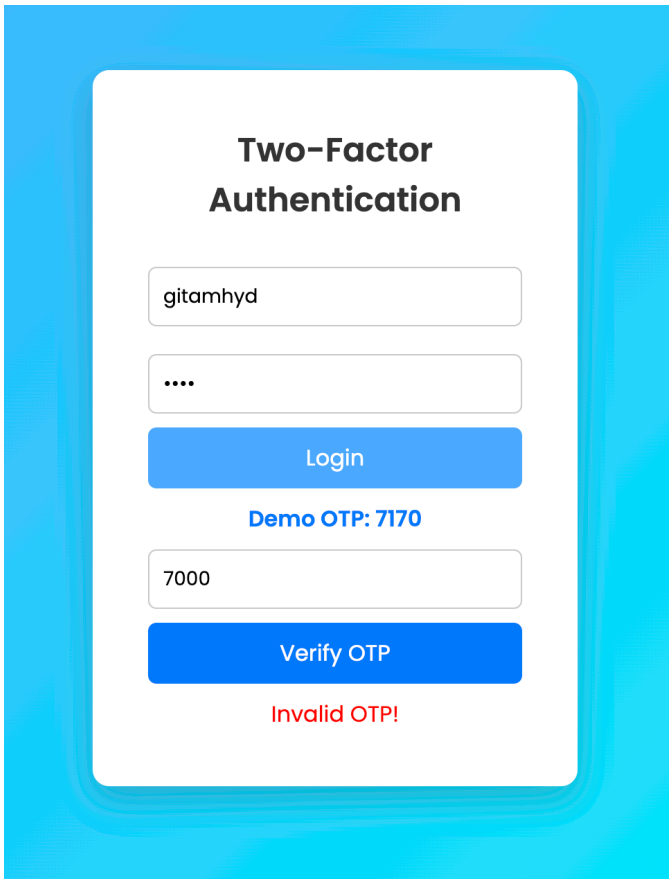
Two-Factor Authentication is widely used in:

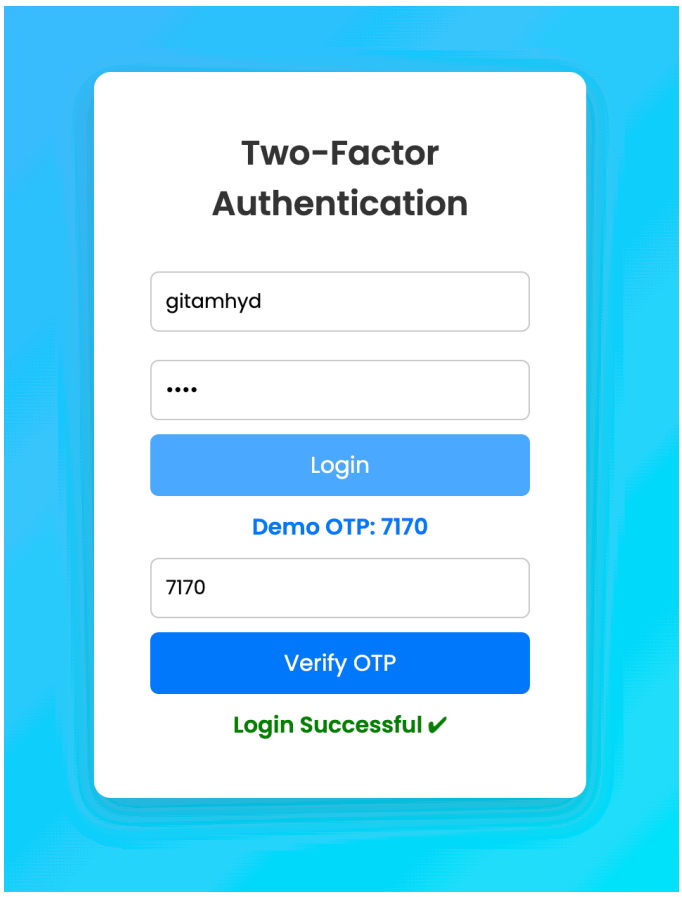
- Online Banking Systems
- Gmail and Email Services
- Social Media Platforms
- Cloud Storage Systems
- Online Payment Applications

Result

The developed model successfully demonstrates the concept of Two-Factor Authentication. The login system requires both password verification and OTP verification before granting access. This shows how adding an extra authentication layer significantly improves system security.







Conclusion

Two-Factor Authentication is an essential cybersecurity technique used to protect online systems from unauthorized access. By combining two authentication factors, systems become more secure and resilient to attacks.

The working model developed in this project successfully demonstrates how OTP-based authentication works. This project helps beginners understand modern security mechanisms used in real-world applications.

References

1. OWASP Security Documentation – https://owasp.org
2. W3Schools JavaScript Guide – https://www.w3schools.com
3. Mozilla Developer Network – https://developer.mozilla.org
4. Google Security Blog – https://security.googleblog.com